

• Section- A (2x5=10)

1. What are the demerits of the SLR parser?

Ans Demerits of the SLR parser -

- Cannot handle left-recursive and ambiguous grammars.
- Fails for grammars requiring more lookahead (conflict in parser table).

2. Define parsing. What is the role of a parser?

Ans Parsing is the process of analyzing a sequence of tokens to determine its grammatical structure.

~~A parser~~ The role of parser to check syntax, build trees and detect errors.

3. Difference b/w backtracking parsing & non-backtracking parsing?

Ans • Backtracking parsing - Explore multiple paths, backtracks if a path fails; computationally expensive.

- Non-Backtracking parsing - Follows a single path, no backtracking; faster but less flexible.

4. What is phrase-level recovery?

Ans it is a error recovery technique in parsing where the parser attempts to correct or continue parsing by adjusting or replacing a small portion (phrases) of the input.

5. What is handle pruning?

Ans it is a technique in bottom-up parsing where the parser identifies and reduces the right-hand side of a production rule (the handle) to its non-terminal symbol.

Purpose :- Ensures the parser focuses on valid reductions, simplifying the parsing process and avoiding unnecessary steps.

• Section-B (7x3=21)

1. Explain the FIRST-FOLLOW procedure.

Ans The FIRST-FOLLOW procedure is used in parsing, particularly in constructing predictive parsers to determine the validity of productions and resolve ambiguities.

• FIRST set :-

The FIRST set of a non-terminal X is the set of terminals that appear at the first symbols in any string derived from X .

• Rules to Compute FIRST(X):

- if X is a terminal, then $\text{FIRST}(X) = \{X\}$
- if $X \rightarrow \epsilon$, then ϵ is in $\text{FIRST}(X)$.
- if $X \rightarrow Y_1 Y_2 \dots Y_n$, then $\text{FIRST}(X)$ includes $\text{FIRST}(Y_1)$ (excluding ϵ), if Y_1 is null, check Y_2 , and so on.

ex: $A \rightarrow aB / \epsilon$
 $B \rightarrow bC / \epsilon$
 $C \rightarrow d / \epsilon$

• $\text{FIRST}(A) = \{a, \epsilon\}$

• $\text{FIRST}(B) = \{b, \epsilon\}$

• $\text{FIRST}(C) = \{d, \epsilon\}$

• FOLLOW set :-

The FOLLOW set of a non-terminal X consists of terminals that can appear immediately after X in some derivation.

• Rules to compute FOLLOW(X):

- if X is the start symbol, add '\$' (end of input) to $\text{FOLLOW}(X)$.
- if $A \rightarrow \alpha X \beta$, then add $\text{FIRST}(\beta)$ (except ϵ) to $\text{FOLLOW}(X)$.
- if $A \rightarrow \alpha X$ or $A \rightarrow \alpha X \beta$ where $\text{FIRST}(\beta)$ contains ϵ , then add $\text{FOLLOW}(A)$ to $\text{FOLLOW}(X)$.

eg: $A \rightarrow AB / \epsilon$
 $B \rightarrow bC / \epsilon$
 $C \rightarrow d / \epsilon$

- Follow(A) = { ϵ }
- Follow(B) = { ϵ }
- Follow(C) = { ϵ }

2. Define handle. Write a short note on handle pruning.

Ans: A handle is a substring of the input string that matches the right-hand side of a production rule in the grammar. It represents the next step in the reduction process during bottom-up parsing.

Handle Pruning is the process of reducing a handle to the non-terminal on the left-hand side of the corresponding production rule during bottom-up parsing. It involves:

- i) Identifying the handle in the stack.
- ii) Replacing the handle with the non-terminal from the production rule.
- (iii) Continuing the parsing process until the start symbol is derived.

Handle Pruning is essential in shift-reduce parsing to ensure the correct derivation of the input string according to the grammar rules. It helps in constructing the parse tree efficiently by reducing the input step-by-step.

2. Explain error Recovery Strategies in Predictive Parsing.

Ans. Error Recovery strategies in predictive parsing are techniques used to handle syntax errors encountered during the parsing process. These strategies allow the parser to continue parsing the input even after detecting an error, rather than stopping abruptly. Here are the key strategies:

(i) Panic mode Recovery :- When an error occurs, the parser skips input tokens until it finds a synchronizing token (e.g. ";" or "{}" in C-like lang.).

ex. `int x = ;` // missing value before '='

• The parser may skip ';' and continue parsing the next statement.

(ii) Phrase-level Recovery - The parser attempts to fix errors by inserting, modifying or deleting tokens so that parsing can continue.

ex. If the input contains:

`if (x > y {` // missing ')' before '{'.

• The parser may insert ')'.

(iii) Error Productions :- The grammar is extended to include common syntax errors so that the parser can provide meaningful error messages.

ex. $E \rightarrow E + T \mid \text{ERROR}$

If a syntax error occurs in an expression, the parser can identify it as an ERROR and handle it accordingly.

(iv) Global Correction :- Uses algorithms to find the minimum changes needed to convert an incorrect input into a valid one.

Advantage - Provides accurate corrections but is computationally expensive.

- Section - C (11x3=33)
- 1. Compute FIRST and FOLLOW of given grammar. Ans
- 2. Create LL(1) Parsing table.
- 3. $E \rightarrow E+T \mid T$
- 4. $T \rightarrow T^*F \mid F$
- 5. $F \rightarrow (E) \mid id$

Ans Step-1: Eliminate Left Recursion.

The given grammar:-

$$E \rightarrow E+T \mid T$$

$$T \rightarrow T^*F \mid F$$

$$F \rightarrow (E) \mid id$$

After Eliminating Left Recursion -

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' \mid \epsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow ^*FT' \mid \epsilon$$

$$F \rightarrow (E) \mid id$$

Formula:

$$\text{if } A \rightarrow A\alpha \mid B$$

$$\text{then: } A \rightarrow B'A'$$

$$A' \rightarrow \epsilon \mid \alpha A'$$

Step 2:- Compute FIRST sets.

The FIRST set of non-terminal is the set of terminals that can appear at the 1st symbol in any string derived from that non-terminal.

i) FIRST (F):

$$\bullet F \rightarrow (E) \Rightarrow \text{FIRST}(F) = \{ '(' \}$$

$$\bullet F \rightarrow id \Rightarrow \text{FIRST}(F) = \{ id \}$$

$$\bullet \text{So } \text{FIRST}(F) = \{ (, id \}$$

ii) FIRST (T'):-

$$\bullet T' \rightarrow ^*FT' \Rightarrow \text{FIRST}(T') = \{ ^* \}$$

$$\bullet T' \rightarrow \epsilon \Rightarrow \text{FIRST}(T') \text{ includes } \epsilon$$

$$\bullet \text{So } \text{FIRST}(T') = \{ ^*, \epsilon \}$$

iii) FIRST (E) :-

- $E' \rightarrow +TE'$ $\Rightarrow$ FIRST (E') = $\{ + \}$
- $E' \rightarrow \epsilon$ $\Rightarrow$ FIRST (E') include ϵ
- So FIRST (E') = $\{ +, \epsilon \}$

(iv) FIRST (E) :-

- $E \rightarrow TE'$ $\Rightarrow$ FIRST (E) = FIRST (T) = $\{ (, id \}$
- So FIRST (E) = $\{ (, id \}$

• Step 3: Compute FOLLOW Sets

The FOLLOW set of non-terminal is the ~~used~~ set of terminals that can appear immediately after the non-terminal, in any sentential form.

i) FOLLOW (E) :-

- E is the start symbol $\Rightarrow \$ \in$ FOLLOW (E)
- $F \rightarrow (E)$ $\Rightarrow) \in$ FOLLOW (E)
- So, FOLLOW (E) = $\{), \$ \}$

ii) FOLLOW (E') :-

- $E \rightarrow TE'$ $\Rightarrow$ FOLLOW (E') = FOLLOW (E) = $\{), \$ \}$
- So, FOLLOW (E') = $\{), \$ \}$

iii) FOLLOW (T) :-

- $E' \rightarrow +TE'$ $\Rightarrow$ FIRST (E') $\subseteq$ FOLLOW (T) = $\{ + \}$
- $E' \rightarrow \epsilon$ $\Rightarrow$ FOLLOW (E') $\subseteq$ FOLLOW (T) = $\{), \$ \}$
- So FOLLOW (T) = $\{ +,), \$ \}$

(iv) FOLLOW (T') :-

- $T \rightarrow *FT'$ $\Rightarrow$ FOLLOW (T') = FOLLOW (T) = $\{ +,), \$ \}$
- So FOLLOW (T') = $\{ +,), \$ \}$

(v) FOLLOW (F) :-

- $T' \rightarrow *FT'$ $\Rightarrow$ FIRST (T') $\subseteq$ FOLLOW (F) = $\{ * \}$
- $T' \rightarrow \epsilon$ $\Rightarrow$ FOLLOW (T') $\subseteq$ FOLLOW (F) = $\{ +,), \$ \}$
- So FOLLOW = $\{ *, +,), \$ \}$

- Step 4: Construct the LR(0) parsing table -
The table is a 2D array where rows represent non-terminals, columns represent terminals, and the entries are LR(0) items.

Non-Terminals	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow id$			$F \rightarrow (E)$		

2. Construct LR(0) parsing table for given grammar:-

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid id$$

- Ans • Step 1: Augment the Grammar.

add a start symbol E' and a production $E' \rightarrow E$ to the grammar.

$$E' \rightarrow E$$

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid id$$

- Step 2: Construct LR(0) items.

Each LR(0) item represents a P.R with a dot (.) indicating how much of the production has been seen.

Closure & GOTO Function.

Now, we compute the closure and GOTO function to create the Canonical Collection of LR(0) items.

• Canonical LR(0) items:-

State	Items
I ₀	$E' \rightarrow \cdot E$ $E \rightarrow \cdot E + T$ $E \rightarrow \cdot T$ $T \rightarrow \cdot T * F$ $T \rightarrow \cdot F$ $F \rightarrow \cdot (E)$ $F \rightarrow \cdot id$
I ₁	$E' \rightarrow E \cdot$
I ₂	$E \rightarrow T \cdot$ $T \rightarrow T \cdot * F$
I ₃	$T \rightarrow F \cdot$
I ₄	$F \rightarrow (\cdot E)$ $E \rightarrow \cdot E + T$ $E \rightarrow \cdot T$ $T \rightarrow \cdot T * F$ $T \rightarrow \cdot F$ $F \rightarrow \cdot (E)$ $F \rightarrow \cdot id$
I ₅	$F \rightarrow id \cdot$
I ₆	$E \rightarrow E \cdot + T$
I ₇	$T \rightarrow T \cdot * F$
I ₈	$F \rightarrow (E \cdot)$
I ₉	$E \rightarrow E + \cdot T$ $T \rightarrow \cdot T * F$ $T \rightarrow \cdot F$ $F \rightarrow \cdot (E)$ $F \rightarrow \cdot id$
I ₁₀	$T \rightarrow T * F \cdot$
I ₁₁	$F \rightarrow (E) \cdot$

* Step 3 :- Forward Left Moving Table
From Table (SHIFT, REDUCE, ACCEPT)

State	id	(	)	+	*	\$	Action
I ₀	SS	SS	-	-	-	-	
I ₁	-	-	ACC	-	-	-	
I ₂	-	-	R2	SE	R2	R2	
I ₃	-	-	R4	R4	R4	R4	
I ₄	SS	SS	-	-	-	-	
I ₅	-	-	R6	R6	R6	R6	
I ₆	SG	-	-	-	-	-	
I ₇	SS	SG	-	-	-	-	
I ₈	-	-	S11	-	-	-	
I ₉	SS	SG	-	-	-	-	
I ₁₀	-	-	R3	R3	R3	R3	
I ₁₁	-	-	R5	R5	R5	R5	

* GOTO Table:-

State	E	T	F
I ₀	1	2	3
I ₁	-	-	-
I ₂	-	-	-
I ₃	-	-	-
I ₄	8	2	3
I ₅	-	-	-
I ₆	-	-	-
I ₇	-	-	-
I ₈	-	-	-
I ₉	-	10	3
I ₁₀	-	-	-
I ₁₁	-	-	-

3. Explain LR Parsers. What are its components?

Ans LR parser (Left-to-right, Rightmost derivation in reverse) are bottom-up parsers used to analyze and process context free grammars. They efficiently parse a large class of grammars, including programming languages.

• Types of LR Parsers :-

(i) LR(0) :-

- The simplest form of LR parsing.
- Uses no lookahead symbols.
- Limited in power and cannot handle many practical grammars.

(ii) SLR(1) (Simple LR) :-

- Enhance LR(0) by using Follow sets to resolve conflicts.
- More powerful than LR(0) but still limited.

(iii) CLR(1) (Canonical LR) :-

- Uses lookahead symbols to resolve conflicts.
- More powerful than SLR(1) but has larger parsing tables.

(iv) LALR(1) (Look-Ahead LR) :-

- A compromise between SLR(1) and CLR(1)
- Combines states in the CLR(1) parsing table to reduce its size while retaining most of its power.

• Components of LR Parsers :- LR parsers consists of the following key components.

(i) Input Buffer -

- stores the string (tokens) to be parsed.
- Symbols are read from left to right.

- (ii) Stack -
• Stack - Stores and maintains symbols.
- (iii) Parsing Table (Action & GOTO Table) -
• Action Table - Determines whether to shift, reduce, accept or reject.
• GOTO Table - Determines state transitions for non-terminals.
- (iv) Driver Program (Parsing Algorithm) -
• Controls the parsing process using the stack and parsing table.
• Repeatedly performs shift (push token to stack) and reduce (replace a set of symbols with a non-terminal) operations.

• Parsing Steps in an LR parser:-

- (i) Shift - Move the next input symbol to the stack.
- (ii) Reduce - Replace a set of symbols with a corresponding grammar rule.
- (iii) Goto - Change the state based on reductions.
- (iv) Accept - Successfully parse the input string.
- (v) Error Handling - Detect & Report syntax errors.

• Advantages:

- (i) Powerful
- (ii) Efficient
- (iii) Error Detection.

• Disadvantages:

- (i) Complexity -
- (ii) Table size.